<!-- SPDX-License-Identifier: GPL-3.0-or-later Copyright (c) 2026 Mohamed Hammad -->

Loran — Implementation Plan v0.1

Field	Value
Project	Loran
Tagline	The Steelbore reference manual.
Document	Implementation Plan
Version	0.1.0 (initial draft)
Date	2026-05-11
Author	Mohamed Hammad
Maintainer	Mohamed Hammad Mohamed.Hammad@Steelbore.com
Copyright	(c) 2026 Mohamed Hammad
License	GPL-3.0-or-later
Website	https://Loran.Steelbore.com/
Governed by	Steelbore Standard v1.1, Steelbore SFRS v1.0.0
PRD	`loran-prd-v0_1.md`
Spec	`loran-spec-v0_2.md`

Table of Contents

---

1. Plan Purpose & How to Read

This Plan operationalises the requirements in `loran-prd-v0_1.md` against the design in `loran-spec-v0_2.md`. It defines:

- **Work packages (WPs)** sized for hobby-pace iteration.
- The **dependency graph** between WPs (what must come before what).
- **Acceptance criteria** per WP.
- The **critical path** through each phase.
- **Cross-cutting workstreams** (CI, docs, benchmarks) that run alongside phase work.
- **Coding conventions** and **release engineering** rules.

Each WP has a header sticker block — phase, sizing, owner crates, inputs, outputs, PRD requirement links, acceptance criteria — followed by a short narrative explaining intent and approach. The ToDo document (next deliverable) decomposes each WP into individual tasks with IDs of the form `LOR-<wp-id>.NNN`.

Reading order for first-time contributors: §2 (Execution Model) → §12 (Critical Path Summary) → §13 (Definition of Done) → drill into the specific phase you're working on.

---

2. Execution Model

2.1 Strict-phase or overlap-permitted?

Strict-phase. Ingot ships before Billet starts. Billet ships before Bloom starts. This is a hobby-pace project; concurrent phases would dilute focus and risk both shipping in worse shape. The cross-cutting workstreams (CI, docs, benchmarks) run alongside all phases.

Within a phase, WPs can be worked in any order that respects their dependency graph. Independent WPs at the same dependency depth are interchangeable.

2.2 Single-maintainer pacing

Per Standard §5.1 (default personal-hobby posture):

- No service-level commitments. Phase completion timelines are intentionally not pinned.
- Each WP is small enough to complete in a focused weekend; no WP is larger than "weekend of dedicated time."

- A WP that grows beyond its sizing during implementation should be split into sub-WPs, not pushed through as-is.

2.3 Definition of "done" for a WP

A WP is done when **all of**:

1. All acceptance criteria are checked off.
2. Code passes `cargo fmt`, `cargo clippy -- -D warnings`, and `cargo test`.
3. Code passes `cargo audit` with no advisories (or all advisories explicitly documented as accepted).
4. SPDX headers on all new source files.
5. Public API items have rustdoc comments.
6. Any new behaviour is reflected in the spec or PRD if it deviates from existing documented behaviour.

2.4 Compliance gates

Three audit moments per release:

- **WP acceptance.** Standard §14 items relevant to the WP must pass.
- **Phase exit.** Full Standard §14 audit must pass before the phase tag.
- **Release.** Full PRD §19 audit + spec compliance section must pass before publishing.

---

3. Sizing Scale & Effort Notation

Relative effort, not calendar time. Hobby pace varies week to week; calendar estimates would be misleading.

Tag	Meaning
XS	A single focused sitting (< 2 hours)
S	A short session (1–2 days of hobby time, ~4–8 focused hours)
M	A weekend project (3–5 sessions, ~10–20 focused hours)
L	Multi-weekend (1–3 weeks of hobby time, ~25–60 focused hours)
XL	Quarter-scale (multiple months of hobby time)

No WP in this plan exceeds **L**. Any task estimated **XL** must be decomposed.

---

4. Phase 0 — Pre-Phase Setup

The first commit's worth of work. Establishes the workspace, posture files, and CI scaffold so subsequent phases have somewhere to land.

WP-P0.01 — Repository initialisation

Phase	Pre-Phase
Sizing	XS
Owner	Repo root
Inputs	—
Outputs	Empty repo with `LICENSE`, `.gitignore`, `.editorconfig`
PRD links	NFR-070, NFR-072

Approach. `git init`, add the GPL-3.0-or-later licence verbatim, basic `.gitignore` (target/, *.bak, .DS_Store), `.editorconfig` (UTF-8, LF, 4-space indent for Rust, 2-space for TOML/Markdown).

Acceptance criteria:

- [ ] `LICENSE` present and verbatim GPL-3.0-or-later
- [ ] `.gitignore` excludes `target/`, IDE cruft, `*.bak`, `node_modules/`
- [ ] `.editorconfig` enforces UTF-8 + LF + appropriate indents
- [ ] Initial commit signed with DCO per CONTRIBUTING.md (which doesn't exist yet, but the discipline starts here)

WP-P0.02 — Posture files (Standard v1.1 §5.2 requirement)

Phase	Pre-Phase
Sizing	S
Owner	Repo root
Inputs	WP-P0.01
Outputs	`README.md`, `NOTICE.md`, `CONTRIBUTING.md`
PRD links	NFR-072, NFR-073

Approach. Author the three posture files per Standard v1.1 §5.2 templates. The README's "Project Posture" section links to NOTICE and CONTRIBUTING. NOTICE carries the no-warranty/no-liability statement deferring to GPL-3.0-or-later. CONTRIBUTING documents PR scope, sign-off (DCO), security-reporting path, license-of-contributions, and the maintainer-discretion principle from Standard §5.4.

Acceptance criteria:

- [ ] README has Project Posture section per Standard §5.1
- [ ] NOTICE carries the canonical no-warranty/no-liability statement
- [ ] CONTRIBUTING covers: PR scope, sign-off, security reporting, license of contributions, maintainer discretion
- [ ] Maintainer attribution per Standard §13.2 in README's "Maintainer" section

WP-P0.03 — Agent context files

Phase	Pre-Phase
Sizing	XS
Owner	Repo root
Inputs	WP-P0.02
Outputs	`AGENTS.md`, `CLAUDE.md`, `SKILL.md`
PRD links	FR-067 (agent context surface)

Approach. AGENTS.md captures repo invariants: Rust-only, GPL-3.0-or-later, SPDX headers required, run `cargo fmt && cargo clippy -- -D warnings && cargo test` before committing, no `unsafe` outside reviewed exceptions, target shells are Nushell and Ion (POSIX sh in scripts). CLAUDE.md adds Claude-Code specific instructions: skills to load (`steelbore-standard`, `steelbore-cli-standard`, `steelbore-agentic-cli`, `rust-guidelines`), reference paths to the spec/PRD/plan. SKILL.md is Loran's own capability surface for the Steelbore Skills system.

Acceptance criteria:

- [ ] AGENTS.md lists every coding-convention invariant
- [ ] CLAUDE.md references the four governing skills and the three governing documents
- [ ] SKILL.md follows Steelbore SKILL format with description and metadata

WP-P0.04 — Cargo workspace skeleton

Phase	Pre-Phase

Sizing	S
Owner	`Cargo.toml`, all crate dirs
Inputs	WP-P0.01
Outputs	Workspace with empty stub crates that build clean
PRD links	NFR-010, NFR-071

Approach. Create the workspace root `Cargo.toml` plus 8 stub crates per spec §3.1: `loran-cli`, `loran-core`, `loran-index`, `loran-pages`, `loran-render`, `loran-tldr`, `loran-tui`, `loran-mcp`. Plus `xtask/`. Each crate has a `lib.rs` (or `main.rs` for cli) with the SPDX header and a single `pub fn placeholder()` stub. Workspace-level `[workspace.dependencies]` table is set up but mostly empty — concrete deps are added per WP.

Acceptance criteria:

- [ ] `cargo build --workspace` succeeds with no warnings
- [ ] Every crate has SPDX header in `Cargo.toml` and root source file
- [ ] Workspace MSRV pinned to a specific stable version in `rust-toolchain.toml`
- [ ] `xtask` runs (even if it just prints "no tasks yet")

WP-P0.05 — Bootstrap CI pipeline

Phase	Pre-Phase
Sizing	M
Owner	`.github/workflows/` or equivalent
Inputs	WP-P0.04
Outputs	Working CI on every push/PR
PRD links	NFR-012

Approach. A single CI workflow that runs on push and PR: `cargo fmt --check`, `cargo clippy -- -D warnings`, `cargo test --workspace`, `cargo audit`, and a license-header sanity check (greps for SPDX on every `.rs` and `Cargo.toml`). Matrix: Linux x86_64 (glibc), Linux x86_64 (musl), Linux aarch64. macOS arm64 added as Tier 2 (allowed to fail without blocking).

Acceptance criteria:

- [ ] CI runs on every push to main and every PR
- [ ] All five checks (fmt, clippy, test, audit, SPDX) gate the build

- [ ] Tier 1 platforms block merge; Tier 2 reports without blocking
- [ ] CI fails fast (parallel jobs, fail-fast not disabled)

---

5. Phase 1 — Ingot (Work Packages)

Phase outcome (from PRD §14.1): A useful binary that lists, shows, finds, and searches the bundled tool catalog, with full JSON output and SFRS-compliant flags. No network, no overlays, no TUI.

WP-P1.01 — Page parser (`loran-pages`)

Phase	Ingot
Sizing	M
Owner crates	`loran-pages`
Inputs	WP-P0.04
Outputs	Parser that ingests a single Markdown+TOML-frontmatter file into a typed `Page` struct
PRD links	FR-035, FR-036, FR-080

Approach. Define the `Page` struct mirroring spec §6.1 schema. Implement the frontmatter splitter (find `+++` fences, parse TOML between them, treat the rest as Markdown body). Use `serde` + `toml` for frontmatter deserialisation; keep the body as raw `String` (no parsing yet — that's `loran-render`'s job). Implement schema validation: required field presence, `safe_alias_for ⊆ replaces` invariant, `summary` ⩽ 120 chars, category-name well-formedness (slash-tolerant). Errors via `thiserror`.

Acceptance criteria:

- [ ] `Page::parse(&str) -> Result<Page, PageError>` works
- [ ] All required fields rejected when absent
- [ ] `safe_alias_for ⊆ replaces` enforced; violation includes the offending name
- [ ] `summary` length check enforced
- [ ] `language` field is parsed but ignored (reserved)
- [ ] Unit tests cover 10+ valid and invalid pages
- [ ] Public API rustdoc-commented

WP-P1.02 — Index loader + `Ingestor` trait (`loran-index`)

Phase	Ingot

Sizing	M
Owner crates	`loran-index`
Inputs	WP-P1.01
Outputs	An in-memory `Index` built from a single source via the `Ingestor` trait
PRD links	FR-070

Approach. Define the `Ingestor` trait with a single method (`fn ingest(&self) -> Result<Vec<Page>, IngestError>`). Implement `MarkdownPagesIngestor` that walks a directory tree and parses every `.md` file via `loran-pages`. The `Index` itself is a value type holding `BTreeMap<String, Page>` keyed by `name`, with secondary indexes (by category, by `replaces` entries, by tags) computed on construction. No postcard cache yet — that's Phase 2 (the cache becomes load-bearing once tarball updates exist). Phase 1 rebuilds the index from bundled sources at every invocation, since bundled sources never change without a rebuild.

Acceptance criteria:

- [ ] `Ingestor` trait defined and documented
- [ ] `MarkdownPagesIngestor` walks a dir and produces a `Vec<Page>`
- [ ] `Index` constructed from `Vec<Page>` with primary + secondary indexes
- [ ] Index build fails loud on any `PageError` (no silent skipping per FR-035)
- [ ] Unit tests cover index construction, lookup, and conflict handling
- [ ] Trait design accommodates `DescribeIngestor` (Phase 3) without API change

WP-P1.03 — Bundled pages tree (build-time)

Phase	Ingot
Sizing	M
Owner crates	`loran-core`, `pages/`
Inputs	WP-P1.01, WP-P1.02
Outputs	A seed catalog of ~25–40 curated pages embedded into the binary via `build.rs`
PRD links	M-01, M-02

Approach. Author curated pages for the Steelbore-canonical tools (eza, bat, rg, fd, procs, bottom, dog, xh, jaq, sd, hyperfine, dust, gitui, helix, nushell, …). Use a `build.rs` script that reads `pages/` at compile time, validates each page via `loran-pages`, and emits a generated `const BUNDLED_PAGES: &[(&str, &str)]` array with relative paths and file contents. The `loran-core`

crate exposes a function to construct a `BundledPagesIngestor` from this constant. Validation in `build.rs` means malformed bundled pages break the build, not the binary at runtime.

Acceptance criteria:

- [ ] `pages/` tree contains 25+ curated pages
- [ ] `build.rs` validates every page via `loran-pages` at compile time
- [ ] Bundled pages cover all 20 legacy tools listed in PRD M-02
- [ ] At least 10 bundled pages include `pairs_with` entries (toward M-03)
- [ ] Bundled pages include `categories.toml`

WP-P1.04 — Core resolution chain (`loran-core`)

Phase	Ingot
Sizing	M
Owner crates	`loran-core`
Inputs	WP-P1.01, WP-P1.02, WP-P1.03
Outputs	The `show` resolution chain per spec §4.1, callable from a single function
PRD links	FR-010, FR-011, FR-012

Approach. Implement `resolve_show(index, tool_name) -> ShowResult`. The result is a typed enum carrying `{IndexHit { intro, body }, NoEntry { hint }, ...}`. In Phase 1 there's no tldr cache so body resolution is custom-or-no-entry only; tldr fallback comes in Phase 2. The function never invokes subprocesses (live `--help` is `loran help`'s job, not `show`'s). Also implement `resolve_find` (reverse lookup) and `resolve_search` (fuzzy match with `nucleo-matcher`).

Acceptance criteria:

- [ ] `resolve_show` returns the correct enum variant for hit/miss
- [ ] `resolve_find` returns all tools whose `replaces` contains the query; `--safe-alias` filter narrows to `safe_alias_for`
- [ ] `resolve_search` does fuzzy match over name/summary/replaces/tags
- [ ] None of these functions panic, even on adversarial input
- [ ] Unit tests cover hit, miss, and edge cases

WP-P1.05 — Live `--help` capture engine (`loran-core`)

Phase	Ingot

Sizing	M
Owner crates	`loran-core`
Inputs	WP-P0.04
Outputs	The `help` capture engine per spec §4.2
PRD links	FR-020 to FR-025, NFR-034

Approach. `capture_help(tool_name) -> HelpResult`. Steps: resolve binary via PATH (using `which` crate or hand-rolled), spawn `Command::new(path).arg("--help")` with no shell, set env (`PAGER="bat -pp"`, `MANPAGER="bat -pp"`, `LESS=""`), enforce 5s timeout via async-or-thread + `kill_on_drop`, capture stdout+stderr, prefer non-empty. On non-zero exit, retry `-h` then `help` subcommand. Returns a typed `HelpResult` with captured text + ISO 8601 UTC capture timestamp + which flag variant succeeded. Implementation uses synchronous threads with a wall-clock timer — keeps the v1 fast path async-free per spec §3.3.

Acceptance criteria:

- [ ] Binary resolution via PATH only; argv never trusted as path
- [ ] argv = [tool, flag] — no shell, no interpolation
- [ ] 5s timeout enforced; SIGKILL on overrun → `LIVE_HELP_TIMEOUT = 9`
- [ ] Retry sequence: `--help` → `-h` → `help`; prefer non-empty
- [ ] Capture timestamp is ISO 8601 UTC with `Z` suffix
- [ ] `bat -pp` env vars set; `cat` fallback if `bat` absent
- [ ] Unit tests use mock binaries (echo scripts) to verify timeout, retry, and capture behaviour

WP-P1.06 — Markdown → terminal text renderer (`loran-render`, text mode)

Phase	Ingot
Sizing	S
Owner crates	`loran-render`
Inputs	WP-P1.01
Outputs	Markdown body → plain-text rendering for non-TTY output
PRD links	NFR-050, NFR-051

Approach. A single function `render_text(body_md, writer) -> Result<()>` using `pulldown-cmark` events. Headings rendered as plain text with capitalisation; code blocks indented 4 spaces; lists bulleted with `-`; links rendered as `[text](url)`. Output is POSIX-parseable. No

ANSI escapes. This is the renderer used when stdout is not a TTY and `--format text` is in effect (or auto-detected). The Steelbore-themed renderer comes in WP-P2.02 alongside the TUI.

Acceptance criteria:

- [ ] Output passes through `grep` / `awk` / `cut` cleanly
- [ ] No ANSI escapes in output
- [ ] Headings, code blocks, lists, links all rendered legibly
- [ ] Unit tests round-trip selected pages

WP-P1.07 — CLI shell with global flags (`loran-cli`)

Phase	Ingot
Sizing	M
Owner crates	`loran-cli`
Inputs	WP-P0.04
Outputs	A `loran` binary that responds to all SFRS §3 global flags; sub-commands are stubs
PRD links	All FR-060 series, NFR-053 to NFR-056

Approach. Define the clap derive structure with every global flag per SFRS §3 (`--json`, `--format`, `--fields`, `--dry-run`, `--verbose`, `--quiet`, `--no-color`, `--color`, `--help`, `--version`, `--absolute-time`, `--print0`, `--yes`). Add sub-command stubs that print "not yet implemented" but accept their flag signatures. `--version` produces the attribution per Standard §13.2 in human mode and the SFRS §6 envelope in JSON mode. `--help` output includes the maintainer footer per Standard §13.2.

Acceptance criteria:

- [ ] All SFRS §3 global flags parse correctly
- [ ] `--version` output passes manual review against Standard §13.2 format
- [ ] `--help` footer matches Standard §13.2
- [ ] `--json --version` returns SFRS §6 envelope with `metadata.maintainer` and `metadata.website`
- [ ] Sub-command stubs accept their full flag surface

WP-P1.08 — JSON envelope (`loran-cli` cross-cutting)

Phase	Ingot
Sizing	S

Owner crates	`loran-cli`
Inputs	WP-P1.07
Outputs	A common output emitter that produces SFRS §6 envelopes
PRD links	FR-060, FR-061, FR-068

Approach. A single module that wraps any "data + metadata" into the SFRS §6 envelope. Provides a `JsonEmitter` type with methods like `emit_data(value)`, `emit_error(code, message, hint)`. Auto-formats timestamps as ISO 8601 UTC with `Z`. Auto-fills `metadata.tool`, `metadata.version`, `metadata.command`, `metadata.maintainer`, `metadata.website`. Used by every sub-command implementation.

Acceptance criteria:

- [ ] Envelope structure matches SFRS §6 exactly
- [ ] Timestamps always carry `Z` suffix (NFR-053)
- [ ] Error envelopes include `error.{code, exit_code, message, hint, timestamp, command}` per SFRS §1 Rule 8
- [ ] Unit tests confirm envelope round-trips through `serde_json`

WP-P1.09 — Agent env-var detection & TTY cascade

Phase	Ingot
Sizing	S
Owner crates	`loran-cli`
Inputs	WP-P1.07, WP-P1.08
Outputs	Auto-detection per SFRS §5: agent env → JSON; non-TTY stdout → JSON; TTY → text (TUI in Billet)
PRD links	FR-061, FR-067

Approach. A module that inspects `AI_AGENT`, `AGENT`, `CI`, `CLAUDECODE`, `CURSOR_AGENT`, `GEMINI_CLI` and `is_terminal()` on stdout. Resolves to one of `{Tui, Text, Json}`. Phase 1 collapses `Tui` to `Text` (TUI doesn't exist yet) and logs a one-line warning to stderr per SFRS §5 when an agent is detected.

Acceptance criteria:

- [ ] Any agent env var → `Json` mode + stderr warning

- [ ] `--json` explicit flag always wins
- [ ] Non-TTY stdout → `Json` mode (per SFRS §5 cascade)
- [ ] TTY stdout → `Text` mode in Phase 1 (will become `Tui` in Phase 2)
- [ ] Detection covered by unit tests with env-var injection

WP-P1.10 — Exit codes + error catalog (`loran-cli`)

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.08
Outputs	An enum `ExitCode` mapping every named error to its numeric code per spec §9
PRD links	FR-068, NFR-051

Approach. Define `ExitCode` enum with variants 0–5 (canonical) and 6–11 (Loran-specific): `INDEX_NOT_BUILT`, `TARBALL_FETCH_FAILED`, `PAGE_PARSE_ERROR`, `LIVE_HELP_TIMEOUT`, `OVERLAY_WRITE_DENIED`, `TARBALL_VERIFY_FAILED`. Each variant has a `hint(context) -> String` method that returns the runnable hint per spec §12.3. Phase 1 only emits a subset (some codes don't apply until Phase 2 features exist), but the full enum is defined for forward compatibility.

Acceptance criteria:

- [ ] All 12 named codes present and documented
- [ ] Every variant has a `hint()` that produces a non-empty string
- [ ] Hints include `--json` variants where appropriate
- [ ] Hints interpolate context (e.g., the user's query in `NOT_FOUND`)

WP-P1.11 — `loran list` sub-command

Phase	Ingot
Sizing	S
Owner crates	`loran-cli`
Inputs	WP-P1.02, WP-P1.07, WP-P1.08

Outputs	Working `loran list` with `--category`, `--replaces`, `--safe-alias-for`, `--fields`
PRD links	FR-001

Approach. Implements `loran list` using the index. Filters by `--category`, `--replaces` (broad match), `--safe-alias-for` (strict match). `--fields name,summary` projects only the requested columns. Default text output: a fixed-width table; JSON output: an array of objects in the SFRS envelope.

Acceptance criteria:

- [ ] All four filter flags work and compose
- [ ] `--fields` projection works
- [ ] Default text output passes POSIX parseability check (NFR-050)
- [ ] JSON output validates against spec §8 example schema
- [ ] Performance: <100ms for catalogs up to 1,000 entries (NFR-002)

WP-P1.12 — `loran show` sub-command

Phase	Ingot
Sizing	S
Owner crates	`loran-cli`
Inputs	WP-P1.04, WP-P1.06, WP-P1.07, WP-P1.08
Outputs	Working `loran show <tool>` per spec §4.1
PRD links	FR-010, FR-011, FR-012

Approach. Dispatches to `loran-core::resolve_show`. On hit, renders intro + body via `loran-render` (text mode); on miss, emits no-entry diagnostic with hint. JSON mode emits the spec §8 envelope. No tldr fallback in Phase 1 (deferred to Phase 2 along with the tldr cache); `body.kind` only takes values `custom` or `none` in Phase 1.

Acceptance criteria:

- [ ] Hit produces intro + body
- [ ] Miss produces the no-entry diagnostic with all hint lines
- [ ] JSON output validates against spec §8
- [ ] `body.kind` correctly reports `custom` or `none`
- [ ] Performance: <50ms cold (NFR-001)

WP-P1.13 — `loran help` sub-command

Phase	Ingot
Sizing	S
Owner crates	`loran-cli`
Inputs	WP-P1.05, WP-P1.07, WP-P1.08
Outputs	Working `loran help <tool>` per spec §4.2
PRD links	FR-020 to FR-025

Approach. Dispatches to `loran-core::capture_help`. Text mode wraps the captured output in a monochrome ASCII frame with the `LIVE OUTPUT —` header; never uses the Steelbore palette. JSON mode emits envelope with `body.kind = "live_help"` and `body.captured_at` per spec §8. On `LIVE_HELP_TIMEOUT`, emits the structured error with hint `loran new <tool> --edit`.

Acceptance criteria:

- [ ] Working capture for a sample binary on PATH
- [ ] Frame is monochrome ASCII — NOT Steelbore palette
- [ ] Header includes ISO 8601 UTC capture timestamp
- [ ] Timeout error envelopes correctly
- [ ] JSON `body.kind = "live_help"` confirmed

WP-P1.14 — `loran find` sub-command

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.04, WP-P1.07
Outputs	Working `loran find <legacy>` with `--safe-alias` filter
PRD links	FR-004, FR-005

Approach. Thin wrapper over `loran-core::resolve_find`. Without `--safe-alias`, returns all entries with `<legacy>` in `replaces`. With `--safe-alias`, restricts to `safe_alias_for` matches. Sort: alphabetical by `name`. Surface fields per spec §8 envelope.

Acceptance criteria:

- [ ] Hits returned correctly for both broad and strict modes
- [ ] Empty results emit clean "no entries supersede X" diagnostic
- [ ] JSON output validates

WP-P1.15 — `loran search` sub-command

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.04, WP-P1.07
Outputs	Working `loran search <query>`
PRD links	FR-003

Approach. Thin wrapper over `loran-core::resolve_search`. Fuzzy match over name/summary/replaces/tags. Result ordering by match score descending.

Acceptance criteria:

- [ ] Fuzzy match returns expected hits for sample queries
- [ ] Ranking is by relevance
- [ ] Empty result set handled cleanly

WP-P1.16 — `loran categories` sub-command

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.02, WP-P1.07
Outputs	Working `loran categories` listing categories with entry counts
PRD links	FR-002

Approach. Read `categories.toml` (bundled) and the index's category index; produce `name | title | count` rows.

Acceptance criteria:

- [ ] Lists all categories from `categories.toml`
- [ ] Counts derived from the live index
- [ ] JSON output includes per-category metadata

WP-P1.17 — `loran describe` sub-command

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.07
Outputs	Self-description manifest per SFRS §4
PRD links	FR-063

Approach. Returns a structured manifest: tool name, version, every sub-command with one-line description, capability tags, link to `loran schema` (which returns "schema available in Phase 3" placeholder for Ingot). Per SFRS §4 schema.

Acceptance criteria:

- [ ] Output validates against SFRS §4 describe schema
- [ ] Every sub-command listed with description
- [ ] Capability tags present (e.g., `read-only`, `network`, `subprocess`)

WP-P1.18 — `loran schema` placeholder

Phase	Ingot
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P1.07
Outputs	A placeholder sub-command that returns a minimal partial schema
PRD links	FR-062

Approach. Phase 1 ships a placeholder that returns the JSON Schema for the `Page` type only, plus a `meta.placeholder: true` field indicating full schema is Phase 3 work. This unblocks

early agent experimentation without committing to the full surface.

Acceptance criteria:

- [ ] Output is valid JSON Schema Draft 2020-12 for the `Page` type
- [ ] `meta.placeholder: true` present
- [ ] Documented in `--help` as "preview; full schema in Bloom phase"

WP-P1.19 — Phase 1 integration test suite

Phase	Ingot
Sizing	M
Owner crates	`loran-cli/tests/`
Inputs	All preceding Phase 1 WPs
Outputs	A test suite covering every sub-command end-to-end
PRD links	All Phase 1 FRs

Approach. Use `assert_cmd` to invoke the built binary against a fixture catalog. Each sub-command gets at least: happy path text, happy path JSON, error path. Snapshot testing via `insta` for stable output formats. Performance tests using `criterion` to guard NFR-001 / NFR-002.

Acceptance criteria:

- [ ] Every sub-command has at least 3 integration tests (happy text, happy JSON, error)
- [ ] Snapshot tests stable across runs
- [ ] Performance tests confirm NFR-001 and NFR-002
- [ ] Test suite runs in CI on every PR

---

6. Phase 2 — Billet (Work Packages)

Phase outcome (from PRD §14.2): The product as users will know it. Full TUI, tarball updates with signature verification, per-distro and per-user overlays, page authoring, schema validation.

WP-P2.01 — Postcard index cache

Phase	Billet
Sizing	S

Owner crates	`loran-index`
Inputs	WP-P1.02
Outputs	A persistent postcard-encoded index cache at `$XDG_CACHE_HOME/loran/index.postcard`
PRD links	NFR-001

Approach. Add a postcard serialization layer to `Index`. On startup, if the cache exists and its timestamp is newer than the latest source modification time, load from cache. Else rebuild and write cache. Rebuild is also forced by `loran update`. This is what gives us <50ms cold-cache `loran show`.

Acceptance criteria:

- [ ] Cache file written atomically (temp + rename)
- [ ] Cache loaded only when fresh
- [ ] Rebuild fallback on any deserialization failure (corrupted cache → rebuild, never panic)
- [ ] Benchmark confirms <50ms cold (NFR-001) — failure here blocks the phase

WP-P2.02 — TUI shell (`loran-tui`)

Phase	Billet
Sizing	M
Owner crates	`loran-tui`
Inputs	WP-P1.07
Outputs	A ratatui app shell that launches, displays a placeholder pane, and exits cleanly
PRD links	FR-006

Approach. `ratatui` + `crossterm`. Event loop, terminal initialisation, panic-safe restoration (terminal restored on Ctrl-C, panic, drop). Theme set up using Steelbore palette tokens from a `loran-render::theme` module. Empty initial app with a placeholder pane and `q` to quit.

Acceptance criteria:

- [ ] Launches on TTY; exits cleanly on `q`, Ctrl-C, panic, signal
- [ ] Terminal state restored on any exit path
- [ ] Palette tokens used (Void Navy bg, Molten Amber text, etc.) per Standard §9
- [ ] `NO_COLOR=1` falls back to terminal default colours

WP-P2.03 — TUI browse view

Phase	Billet
Sizing	M
Owner crates	`loran-tui`
Inputs	WP-P2.02, WP-P1.02
Outputs	Dual-pane browser: categories left, tools right
PRD links	FR-006

Approach. Two-pane layout. Left pane: list of categories from `categories.toml` with counts. Right pane: list of tools in the selected category, with name + one-line summary. Tab switches focus between panes; `j`/`k` (Vim) and arrow keys (CUA) navigate; Enter on a tool opens the detail view (WP-P2.04). `/` opens fuzzy-search overlay.

Acceptance criteria:

- [ ] Both panes populated from live index
- [ ] Vim and CUA keybindings work in both panes
- [ ] Tab focus switching visible
- [ ] Empty category shows "no tools" placeholder

WP-P2.04 — TUI detail view

Phase	Billet
Sizing	M
Owner crates	`loran-tui`, `loran-render`
Inputs	WP-P2.03, WP-P1.06
Outputs	Rendered Markdown body with Steelbore intro, pairs/safe-alias badges, frontmatter side-panel
PRD links	FR-013, FR-014, FR-015

Approach. Add a Markdown-to-`ratatui-text` renderer for the body. Top section: Steelbore intro block. Middle: rendered body. Right sidebar: badges for `pairs_with`, `safe_alias_for`,

`written_in` (with 🦀 for rust). Tab cycles through detail, raw Markdown, and frontmatter views (agent-friendly inspection per spec §10).

Acceptance criteria:

- [ ] Body renders headings, code blocks, lists, links correctly
- [ ] Steelbore palette applied
- [ ] `pairs_with` sidebar populated
- [ ] `safe_alias_for` badge distinct from `replaces`
- [ ] 🦀 badge appears for `written_in = "rust"`
- [ ] Tab cycles through three views

WP-P2.05 — TUI fuzzy search overlay

Phase	Billet
Sizing	S
Owner crates	`loran-tui`
Inputs	WP-P2.03, `nucleo-matcher`
Outputs	A `/`-triggered search overlay that filters the catalog as you type
PRD links	FR-003

Approach. Modal input overlay. Live filtering via `nucleo-matcher`. Result list updates per keystroke. Enter selects + opens detail view; Esc cancels and restores prior view.

Acceptance criteria:

- [ ] `/` enters search mode
- [ ] Live filter updates as you type
- [ ] Enter opens result; Esc cancels
- [ ] Latency from keystroke to render <16ms (NFR-005)

WP-P2.06 — TUI in-app help

Phase	Billet
Sizing	XS
Owner crates	`loran-tui`
Inputs	WP-P2.02

Outputs	`?` opens a help overlay listing keybindings
PRD links	NFR-062

Approach. Static help overlay. Lists both CUA and Vim bindings side by side. Esc dismisses.

Acceptance criteria:

- [ ] `?` opens overlay
- [ ] Both keybinding schemes listed
- [ ] Esc dismisses

WP-P2.07 — HTTP client + manifest fetch

Phase	Billet
Sizing	S
Owner crates	`loran-tldr`
Inputs	`ureq`, `rustls`
Outputs	A wrapper for fetching with `If-None-Match` + ETag caching
PRD links	FR-040, FR-041

Approach. Thin `ureq` wrapper. Sends `If-None-Match` against cached ETag. Returns `Fetched(bytes)`, `NotModified`, or `Err(...)`. Stores ETag + last-modified in `meta.toml` alongside the cache file.

Acceptance criteria:

- [ ] Fetches and returns body
- [ ] 304 NotModified handled correctly
- [ ] Network errors (DNS, TLS, timeout) mapped to `TARBALL_FETCH_FAILED = 7`
- [ ] Exponential backoff with 3 retries per NFR-023

WP-P2.08 — Tar/gzip extraction (atomic)

Phase	Billet
Sizing	S
Owner crates	`loran-tldr`

Inputs	`tar`, `flate2`
Outputs	An extractor that takes a `.tar.gz` and writes atomically to a target directory
PRD links	FR-045, NFR-020

Approach. Extract into `$XDG_DATA_HOME/loran/pages.tmp/` (or `$XDG_CACHE_HOME/loran/tldr.tmp/`). On success, `rename` the temp dir to the live location. On failure, delete the temp dir and leave the live location untouched. Per-file checks (size limits, path traversal sanitisation) before any write.

Acceptance criteria:

- [ ] Atomic swap via `rename`
- [ ] Path traversal protection (no `..` or absolute paths in tarball)
- [ ] Failure leaves live state intact
- [ ] Test: deliberately corrupted tarball never corrupts the live tree

WP-P2.09 — Minisign verification

Phase	Billet
Sizing	S
Owner crates	`loran-tldr`
Inputs	`minisign-verify`, WP-P2.07
Outputs	Signature verification gate that runs before extraction
PRD links	FR-043, FR-044, NFR-030, NFR-031

Approach. Bake the publisher's ed25519 public key into the binary via `include_bytes!`. Fetch `pages.tar.gz.minisig` alongside the tarball. Verify with `minisign-verify::verify`. On failure, exit `TARBALL_VERIFY_FAILED = 11` with hint. The key bytes themselves live in `loran-tldr/keys/pages.pub` and are committed alongside the source.

Acceptance criteria:

- [ ] Public key embedded via `include_bytes!`
- [ ] Valid signature passes
- [ ] Tampered tarball fails with exit 11
- [ ] Wrong-key signature fails with exit 11
- [ ] No extraction attempted on verify failure
- [ ] Test fixtures include valid signed and tampered tarballs

WP-P2.10 — Upstream pages tarball pipeline (client side)

Phase	Billet
Sizing	S
Owner crates	`loran-tldr` (or new module)
Inputs	WP-P2.07, WP-P2.08, WP-P2.09
Outputs	End-to-end flow: fetch manifest → fetch tarball + sig → verify SHA → verify sig → extract → rebuild index
PRD links	FR-040 to FR-046

Approach. Composes the previous three WPs into a single `update_upstream_pages()` flow. Failure at any step preserves the existing catalog. Success rebuilds the postcard index automatically.

Acceptance criteria:

- [ ] All 6 verification steps in correct order per spec §11
- [ ] Index rebuilt after successful extract
- [ ] `--dry-run` reports without touching disk
- [ ] Each failure mode produces the correct exit code and hint

WP-P2.11 — tldr-pages tarball fetch

Phase	Billet
Sizing	XS
Owner crates	`loran-tldr`
Inputs	WP-P2.07, WP-P2.08
Outputs	Separate flow for tldr fetch — SHA-256 only, no signature
PRD links	FR-047, FR-048

Approach. Same shape as WP-P2.10 but with the signature step omitted (tldr-pages upstream doesn't sign). `--require-signatures` makes the tldr fetch refuse entirely with a clear diagnostic.

Acceptance criteria:

- [ ] tldr tarball fetched and extracted under `$XDG_CACHE_HOME/loran/tldr/`
- [ ] SHA-256 verification gates extraction
- [ ] `--require-signatures` refuses tldr fetch with `TARBALL_VERIFY_FAILED` and explanatory hint
- [ ] tldr fetch failure is non-fatal to the rest of the update

WP-P2.12 — `loran update` sub-command wiring

Phase	Billet
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P2.10, WP-P2.11
Outputs	`loran update` sub-command end-to-end
PRD links	FR-040 to FR-049

Approach. Wire the upstream + tldr flows behind the `update` sub-command. Verbose mode logs each step to stderr; quiet mode suppresses all but errors. `--dry-run` reports without touching disk.

Acceptance criteria:

- [ ] Runs upstream + tldr fetches in sequence
- [ ] `--dry-run` honoured
- [ ] `--verbose` and `--quiet` modes behave correctly
- [ ] Integration test covers signed + tampered + dry-run scenarios

WP-P2.13 — Overlay merge engine

Phase	Billet
Sizing	M
Owner crates	`loran-index`
Inputs	WP-P1.02
Outputs	Three-layer overlay merge: upstream → distro → user
PRD links	FR-050, FR-051, FR-052, FR-053, FR-054

Approach. Extend the index loader to walk three roots (upstream `pages/`, `overlays/<active>/`, `overlays/user/`) in precedence order. Merge field-by-field at the `Page` struct level: later layers override earlier ones, but only for fields they explicitly set. Active distro resolved from `/etc/os-release`. `--overlay <name>` flag overrides. User overlay may add categories but not remove upstream ones.

Acceptance criteria:

- [ ] Three-layer merge implemented per spec §5.1
- [ ] Field-level override (user can change `summary` without re-stating `category`)
- [ ] `/etc/os-release` parsing handles `ID=bravais`, `ID=ferrite`, and falls back to "generic"
- [ ] `--overlay <name>` flag works
- [ ] User overlay cannot remove upstream categories (rejected with `OVERLAY_WRITE_DENIED` hint)
- [ ] Unit tests cover override precedence, additive categories, and conflict cases

WP-P2.14 — Page template + `loran new` (non-interactive)

Phase	Billet
Sizing	S
Owner crates	`loran-cli`
Inputs	WP-P2.13
Outputs	Working `loran new` in non-interactive mode
PRD links	FR-030, FR-031, FR-032, FR-034

Approach. Default template baked into binary; copied to `$XDG_DATA_HOME/loran/templates/tool.md` on first run if absent. Non-interactive mode accepts every required + optional field as a flag, validates inputs, writes to `overlays/user/<category>/<tool>.md`. `--scope=upstream` redirects to the user-configured upstream checkout path.

Acceptance criteria:

- [ ] Default template ships in binary
- [ ] Template copied to XDG path on first run
- [ ] Non-interactive flags accepted and validated
- [ ] File written atomically (temp + rename)
- [ ] `--scope=upstream` redirects correctly
- [ ] `OVERLAY_WRITE_DENIED = 10` on permission failure

WP-P2.15 — `loran new` interactive mode

Phase	Billet
Sizing	M
Owner crates	`loran-cli`, `loran-tui`
Inputs	WP-P2.14, WP-P2.02
Outputs	Working `loran new` in interactive mode
PRD links	FR-033

Approach. When stdout is a TTY and no flags are supplied, drop into a small ratatui prompt sequence: category (autocomplete from `categories.toml`), summary, replaces. Then write the scaffold and shell out to `$EDITOR`.

Acceptance criteria:

- [ ] Interactive prompt sequence works on TTY
- [ ] Category autocomplete from `categories.toml`
- [ ] `$EDITOR` invoked on body afterwards (configurable via `--no-edit`)
- [ ] Cancel (Esc/Ctrl-C) leaves no partial file behind

WP-P2.16 — `loran validate` sub-command

Phase	Billet
Sizing	S
Owner crates	`loran-cli`, `loran-pages`
Inputs	WP-P1.01, WP-P2.13
Outputs	Working `loran validate`
PRD links	FR-035, FR-036

Approach. Walks all three overlay roots and validates every page via `loran-pages`. Reports errors with file path + line number. Exits non-zero on any error. JSON mode emits a structured list of `{file, line, code, message}` entries.

Acceptance criteria:

- [ ] Validates every page, not just one

- [ ] File path + line number in every error
- [ ] CI-friendly: exit non-zero on any error
- [ ] JSON output schema documented

WP-P2.17 — tldr fallback in resolution chain

Phase	Billet
Sizing	S
Owner crates	`loran-core`
Inputs	WP-P2.11, WP-P1.04
Outputs	`resolve_show` falls through to tldr when no custom page exists
PRD links	FR-011

Approach. Extend `resolve_show`: after checking for custom page in all overlays, look up `tldr_page` (or default to `name`) in the cached tldr tree. If found, return as `body.kind = "tldr"`. If not, return `body.kind = "none"`.

Acceptance criteria:

- [ ] Custom page wins when both custom and tldr exist
- [ ] tldr used as fallback when no custom page
- [ ] No-entry diagnostic when neither
- [ ] `body.kind` reports the correct variant
- [ ] `tldr_page = ""` in frontmatter disables tldr lookup

WP-P2.18 — Phase 2 integration test suite

Phase	Billet
Sizing	M
Owner crates	`loran-cli/tests/`, `tests/`
Inputs	All preceding Phase 2 WPs
Outputs	End-to-end tests for TUI, update, overlays, authoring
PRD links	All Phase 2 FRs

Approach. Headless TUI tests via `ratatui::backend::TestBackend`. Update-flow tests with fixture tarballs (valid + tampered + missing-sig). Overlay tests with fixture trees. Authoring tests with `assert_cmd` + temp dirs.

Acceptance criteria:

- [ ] TUI tests cover browse, search, detail views via TestBackend
- [ ] Update tests cover all signature + checksum failure modes
- [ ] Overlay tests cover precedence, field-merge, additive-category cases
- [ ] Authoring tests cover interactive + non-interactive + cancel cases
- [ ] CI runs the full suite on every PR

---

7. Phase 3 — Bloom (Work Packages)

Phase outcome (from PRD §14.3): Loran becomes the primary tool-discovery surface for AI agents on Steelbore systems, and the ecosystem becomes self-documenting via SFRS describe ingestion.

WP-P3.01 — Full JSON Schema emission

Phase	Bloom
Sizing	M
Owner crates	`loran-cli`, `loran-pages`, `loran-core`
Inputs	Phase 2 complete
Outputs	`loran schema` emits full JSON Schema Draft 2020-12 of all public types
PRD links	FR-062

Approach. Replace the Phase 1 placeholder with a full schema covering: `Page`, `ListEntry`, `SearchResult`, `Category`, error envelope, the `Show` and `Help` body variants, and the `describe` manifest. Use `schemars` crate to derive schemas from Rust types directly so they stay in sync. Output is a single JSON document; sub-schemas are linked via `$ref`.

Acceptance criteria:

- [ ] Full schema validates against JSON Schema Draft 2020-12 meta-schema
- [ ] Every public type has a schema entry
- [ ] `$ref` references resolve correctly
- [ ] Tested against Anthropic, OpenAI, Gemini function-calling format expectations

WP-P3.02 — MCP server crate (`loran-mcp`)

Phase	Bloom
Sizing	M
Owner crates	`loran-mcp`
Inputs	WP-P3.01, `rmcp`
Outputs	An MCP server over stdio exposing read-only verbs
PRD links	FR-064, FR-065

Approach. `rmcp`-based server. Implements `initialize`, `tools/list`, `tools/get`, `tools/call` for the read-only verbs (`list`, `show`, `find`, `search`, `categories`). `tools/list` advertises names + capability tags only; full schemas come from `tools/get` per lazy-loading discipline. `tools/call` dispatches to the same `loran-core` resolve functions used by the CLI. No async runtime in the rest of the workspace — async is contained to this crate.

Acceptance criteria:

- [ ] MCP server starts and responds to `initialize`
- [ ] `tools/list` returns only the 5 read-only verbs
- [ ] `tools/get` returns the schema for a specific verb
- [ ] `tools/call` produces SFRS-compliant envelopes
- [ ] Attempts to call `update`, `new`, `validate`, `help` via MCP are rejected
- [ ] Integration test via `rmcp` test harness

WP-P3.03 — `loran mcp` sub-command wiring

Phase	Bloom
Sizing	XS
Owner crates	`loran-cli`
Inputs	WP-P3.02
Outputs	`loran mcp` starts the MCP server over stdio
PRD links	FR-064

Approach. Thin sub-command that hands off to `loran-mcp::serve(stdin, stdout)`. Logs go to stderr only.

Acceptance criteria:

- [ ] `loran mcp` starts and serves
- [ ] No stdout output except MCP protocol traffic
- [ ] stderr carries structured logs per SFRS

WP-P3.04 — `DescribeIngestor` implementation

Phase	Bloom
Sizing	M
Owner crates	`loran-index`
Inputs	WP-P1.02 (Ingestor trait)
Outputs	A new `Ingestor` impl that spawns `<tool> describe --json` against allowlisted Steelbore binaries
PRD links	FR-071, FR-072, FR-073

Approach. Implements `DescribeIngestor`. Allowlist of trusted binary names lives in the upstream pages tarball as `trusted_describe.toml`. For each name, resolve via `$PATH`, spawn with sandbox per WP-P1.05's invariants, parse SFRS-§4 describe JSON, synthesise a baseline `Page`. Curated pages overlay on top.

Acceptance criteria:

- [ ] Allowlist sourced from `trusted_describe.toml`
- [ ] Spawn uses the same sandbox as `loran-core::capture_help`
- [ ] SFRS-compliant describe output parsed correctly
- [ ] Baseline page generated with `name`, `summary`, `category` (from describe metadata), `replaces` (if declared)
- [ ] Curated pages always override `DescribeIngestor` output

WP-P3.05 — Minisign key rotation documentation

Phase	Bloom
Sizing	S
Owner	`OPERATIONS.md`, repo root
Inputs	WP-P2.09

Outputs	A documented procedure for rotating the publisher's signing key
PRD links	Open Question 1

Approach. Authors OPERATIONS.md covering: (a) the trust-pinned-key constraint, (b) the normal rotation cadence (annual? when compromise suspected?), (c) parallel-key transition (multiple `include_bytes!` keys for a transition window), (d) emergency rotation procedure (compromise → new release with new key → revoke old). Resolves PRD Open Question 1.

Acceptance criteria:

- [ ] OPERATIONS.md authored
- [ ] Normal-rotation procedure documented
- [ ] Parallel-key transition mechanism specified and implemented in `loran-tldr`
- [ ] Emergency-rotation procedure documented
- [ ] Spec §15 and PRD §18 updated to mark Open Question 1 as resolved

WP-P3.06 — Cross-distro overlay surfacing

Phase	Bloom
Sizing	S
Owner crates	Tarball pipeline (separate project) + `loran-tldr`
Inputs	WP-P2.10
Outputs	The upstream tarball includes Bravais + Ferrite overlays alongside the generic catalog
PRD links	PRD §13.1

Approach. Coordinates with the publisher pipeline (separate concern) to bundle per-distro overlays into the upstream tarball. `loran-tldr` already extracts overlays into the right paths via WP-P2.08; this WP is about content provenance and pipeline coordination.

Acceptance criteria:

- [ ] Published tarball contains `overlays/bravais/` and `overlays/ferrite/` subdirs
- [ ] Client extraction populates the corresponding overlay roots
- [ ] Bravais overlay is sourced from the Bravais repo (publisher contract)
- [ ] Ferrite overlay is sourced from the Ferrite OS repo (publisher contract)

WP-P3.07 — Phase 3 integration test suite

Phase	Bloom

Sizing	M
Owner crates	`tests/`
Inputs	All preceding Phase 3 WPs
Outputs	End-to-end tests for MCP, schema emission, DescribeIngestor
PRD links	All Phase 3 FRs

Approach. MCP tests via `rmcp` test harness invoking every read-only verb and confirming write verbs are rejected. Schema tests validate against the JSON Schema Draft 2020-12 meta-schema. DescribeIngestor tests with mock Steelbore-style describe-output binaries.

Acceptance criteria:

- [ ] MCP test suite covers all 5 read-only verbs + rejection of write verbs
- [ ] Schema validates against meta-schema in CI
- [ ] DescribeIngestor tests cover allowlist, sandbox enforcement, override precedence
- [ ] CI runs the full Phase 3 suite on every PR

---

8. Cross-Cutting Workstreams

These run alongside all phases. None has a fixed phase; each starts in Phase 0 and continues through Phase 3 and beyond.

WP-CC.01 — CI pipeline maintenance

Phase	All
Sizing	Ongoing
Owner	`.github/workflows/` (or equivalent)
Inputs	WP-P0.05
PRD links	NFR-012

Approach. Initial setup in WP-P0.05; ongoing maintenance is part of every WP that introduces new tooling. Add per-platform jobs as Tier 1 expands. Track CI duration; if a single run exceeds 10 minutes, optimise or split.

Ongoing acceptance criteria:

- [ ] CI duration < 10 minutes

- [ ] Tier 1 platforms gate merge
- [ ] No flaky tests tolerated (flaky → fix or quarantine)

WP-CC.02 — Benchmark suite

Phase	All
Sizing	S (initial), then ongoing
Owner	`loran-cli/benches/`, `xtask`
PRD links	NFR-001 to NFR-005, M-05, M-06

Approach. `criterion`-based bench suite. Initial benches in Phase 1 for `loran show` cold and `loran list`. Phase 2 adds index-rebuild and tarball-extraction benches. Phase 3 adds MCP roundtrip bench. Every release runs the suite; regressions block release per M-06.

Acceptance criteria:

- [ ] Bench suite exists from end of Phase 1
- [ ] All NFR-001 to NFR-005 covered
- [ ] Regression detection in CI (fail on > 10% slowdown)
- [ ] Benchmarks documented in BENCHMARKS.md

WP-CC.03 — Documentation maintenance

Phase	All
Sizing	Ongoing
Owner	Repo root + per-crate docs
PRD links	NFR-072

Approach. Every WP touching public API updates rustdoc. README updated when user-visible behaviour changes. CHANGELOG kept in sync with every released version. AGENTS.md and CLAUDE.md updated when conventions evolve. Spec and PRD are updated when design or requirements change; version bumps trigger PRD/spec amendments.

Acceptance criteria:

- [ ] No public API item ships without rustdoc
- [ ] README reflects shipped behaviour, not aspirational
- [ ] CHANGELOG accurate
- [ ] Spec, PRD, plan kept in sync at major version transitions

WP-CC.04 — Page authoring (content)

Phase	Phase 1 onwards, ongoing
Sizing	Continuous
Owner	`pages/` (this repo) + per-distro repos
PRD links	M-01, M-02, M-03

Approach. Page authoring is distinct from page tooling. Initial seed catalog (Phase 1, ~25–40 pages) covers the most-frequently-used legacy-replacement tools (eza, bat, rg, fd, procs, bottom, dog, xh, jaq, etc.). Phase 2 adds another 50+ to bring Bravais default-install coverage above 80% (M-01). Phase 3 leans on `DescribeIngestor` to auto-cover Steelbore-native CLIs.

Acceptance criteria:

- [ ] Phase 1 ships with 25+ curated pages
- [ ] Phase 2 ships with ⩾80% Bravais default-install coverage
- [ ] All 20 legacy tools in PRD M-02 have at least one entry by Phase 2

WP-CC.05 — Security review

Phase	All
Sizing	Per-release
Owner	Maintainer
PRD links	NFR-012, NFR-030 to NFR-035

Approach. Pre-release: run `cargo audit`, review new dependencies added since last release, confirm no new `unsafe` blocks, confirm minisign verification still gates updates, confirm `loran help` sandbox invariants. Dependency additions in PRs trigger a mini-review (license + maintenance status).

Acceptance criteria:

- [ ] `cargo audit` clean before every release tag
- [ ] New dependencies justified in PR descriptions
- [ ] No `unsafe` blocks introduced without review note
- [ ] Sandbox invariants documented and verified per release

WP-CC.06 — Release engineering

Phase	All
Sizing	Per-release
Owner	Maintainer + CI
PRD links	NFR-031, NFR-070

Approach. Detailed in §10 below.

WP-CC.07 — Publisher pipeline coordination

Phase	Phase 2 onwards
Sizing	M (initial), then ongoing
Owner	Separate project, coordinated by maintainer
PRD links	§12.1

Approach. The upstream tarball is produced by a separate publisher pipeline (out of scope for the Loran binary). Coordinating this is a cross-cutting workstream: define the contract (manifest format, tarball structure, signing requirements), build the publisher tooling (separate repo), and ensure the Bravais and Ferrite repos can push their overlays into the publishing pipeline.

Acceptance criteria:

- [ ] Publisher contract documented in a separate `PUBLISHING.md`
- [ ] Publisher repo exists by start of Phase 2
- [ ] First signed tarball published before Billet ships

WP-CC.08 — Compliance audits

Phase	Per-phase + per-release
Sizing	XS each
Owner	Maintainer
PRD links	PRD §19, Standard §14

Approach. Run through Standard §14 checklist at: WP acceptance, phase exit, every release. Document the result in a `compliance-log.md` file with date + auditor + items + status. Flag any deviation immediately.

Acceptance criteria:

- [ ] Audit log exists from end of Phase 0
- [ ] Every phase exit + every release recorded
- [ ] Deviations annotated and either fixed or formally accepted (Standard §5.4)

---

9. Workspace Engineering Standards

9.1 Crate organisation

- Workspace root `Cargo.toml` declares `[workspace.dependencies]` with pinned versions; member crates use `dep = { workspace = true }`.
- No cyclic dependencies between member crates. `loran-cli` depends on everything; `loran-core` depends on `loran-pages` + `loran-index`; everything else is a leaf or has dependencies only on more fundamental crates.
- Public APIs have stability commitments only for `loran-cli` (the binary's CLI surface). Internal crates can break between minor versions.

9.2 Error handling

- Library crates use `thiserror` for typed errors.
- Binary crate (`loran-cli`) uses `anyhow` at the top level and translates to typed errors at function boundaries.
- Every error type that crosses a process boundary (i.e., is reported to the user) has a stable `code`, a human message, and a runnable `hint` per SFRS tips-thinking discipline.

9.3 Logging & tracing

- `tracing` for all logs; `tracing-subscriber` configured in `loran-cli`.
- Log levels: ERROR (user-visible failures), WARN (degraded behaviour), INFO (lifecycle events), DEBUG (development). Never use `println!` or `eprintln!` for diagnostics.
- All logs go to stderr; stdout is data only per SFRS §1 Rule 7.
- Structured logs in JSON when `--format json` is active.

9.4 Time handling

- Per Standard §12.5: `jiff::Timestamp` for all stored values.
- Never serialise with offsets; always `Z` suffix.
- Internal computations may use `std::time::Instant` for monotonic measurements; conversions to wall-clock go through `jiff`.

9.5 Unsafe code policy

- Default: no `unsafe` blocks. Any introduction requires a code comment block explaining why and what invariants are upheld.
- Crate-level `#![deny(unsafe_code)]` on every crate except those with documented exceptions.
- `#![forbid(unsafe_code)]` on `loran-pages`, `loran-render`, `loran-core` (these have no plausible unsafe need).

9.6 Test policy

- Unit tests live alongside source (`#[cfg(test)] mod tests {...}`).
- Integration tests live in `tests/` per crate.
- End-to-end tests use `assert_cmd` to invoke the built binary.
- Snapshot tests via `insta`.
- Performance tests via `criterion`.
- Every WP ships its own tests; tests are part of the WP, not a follow-up.

9.7 Code style

- `rustfmt` defaults plus `imports_granularity = "Crate"` (configured in `rustfmt.toml`).
- `clippy::pedantic` enabled selectively; full `pedantic` is too noisy for this codebase but `correctness` + `suspicious` + `style` are non-negotiable.
- `cargo clippy -- -D warnings` is the gate.

9.8 Commit & PR discipline

- Conventional Commits (`feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`).
- Every commit includes the DCO sign-off (per CONTRIBUTING.md).
- PRs reference the WP-ID and the relevant FR/NFR IDs in their description.
- PRs squashed to a single commit on merge.

9.9 Shell scripting conventions (xtask + tooling)

- `xtask` is a Rust binary, not shell scripts. Cross-platform, no Bash-isms.
- Where shell scripts are unavoidable (CI snippets, release scripts), POSIX `sh` only.
- User-facing examples in docs use the user's shell (Nushell, Ion, or POSIX sh depending on context per the user's preferences). Never assume Bash.

---

10. Release Engineering

10.1 Versioning

Semantic versioning per https://semver.org. The version is the `loran-cli` binary's CLI surface plus the JSON schema. Breaking changes to either bump the major.

Pre-1.0 (Phase 1, early Phase 2): every minor version may break the JSON schema. Documented loudly in the changelog.

After Billet ships at v1.0: only major versions may break the JSON schema. Deprecation cycle: announce in version N, support in N and N+1, remove in N+2 (where major versions of Loran are typically separated by months of hobby time, not minutes).

10.2 Release artefacts

Per release tag:

- Source tarball (`.tar.gz`)
- Pre-built binaries: Linux x86_64 (glibc), Linux x86_64 (musl), Linux aarch64, FreeBSD amd64
- SHA-256 manifest
- Minisign signatures for every artefact (separate key from the pages-tarball signing key)
- CHANGELOG entry

10.3 Release process

1. Confirm CI is green on the target commit.
2. Run `cargo audit`.
3. Run the benchmark suite; confirm no NFR regressions.
4. Run the §14 compliance checklist.
5. Update CHANGELOG.
6. Bump version in workspace `Cargo.toml`.
7. Tag the commit.
8. CI builds and signs artefacts.
9. Publish release with artefacts + signatures.
10. Update the `compliance-log.md` with the release audit result.

10.4 Hot-fix release process

For security fixes only:

1. Branch from the last release tag.
2. Apply the fix.
3. Run the full release process (including audit) on the branch.

4. Release as a patch version.
5. Forward-port the fix to main.

---

11. Risk-Mitigation Workstreams

These are not phase-aligned; they are ongoing concerns that demand explicit attention.

11.1 Curation-burden mitigation

Risk: Catalog growth outpaces curation quality.

Workstream actions:

- Schema validation (`loran validate`) blocks malformed pages.
- A style guide in `CONTRIBUTING.md` defines what "good" looks like.
- Phase 3's `DescribeIngestor` reduces from-zero authoring cost.
- Page reviews track a quality metric (curated body length, presence of `pairs_with`, presence of examples).

11.2 Key-compromise mitigation

Risk: Publisher signing key is compromised.

Workstream actions:

- Hardware-token storage of the publisher key in normal operation.
- Documented emergency rotation procedure (WP-P3.05).
- Parallel-key transition windows in `loran-tldr`.
- Annual key-rotation drills (post-Bloom).

11.3 Upstream-breakage mitigation

Risk: tldr-pages or the Rust crate ecosystem changes in incompatible ways.

Workstream actions:

- tldr fallback is non-fatal — Loran works without it.
- Dependency-pinning at the workspace level (no `*` versions).
- `cargo audit` gates releases.
- Test fixtures for the tldr tarball shape ensure breakage is detected early.

11.4 Bus-factor mitigation

Risk: Single-maintainer project; absence stalls everything.

Workstream actions:

- License (GPL-3.0-or-later) permits forks.
- All decisions documented (spec + PRD + plan + compliance log).
- Style guide and CONTRIBUTING.md lower contribution friction.
- No proprietary tooling in the build chain.

---

12. Critical Path Summary

The minimum dependency chain through each phase. Anything off the critical path can be parallelised; anything on it is a serial blocker.

12.1 Phase 1 critical path (Ingot)

```
WP-P0.04 (workspace skeleton)
    → WP-P1.01 (page parser)
        → WP-P1.02 (index loader)
            → WP-P1.03 (bundled pages)
                → WP-P1.04 (resolution chain)
                    → WP-P1.12 (loran show)
                        → WP-P1.19 (integration test suite)
                            = Ingot ready to tag
```

Off the critical path but required for Ingot: WP-P0.01, P0.02, P0.03, P0.05 (setup), WP-P1.05 (help capture, can run in parallel with index work), WP-P1.06 (renderer, parallel with show), WP-P1.07 to P1.10 (CLI shell + envelope + agent detection + exit codes — these gate all sub-commands so they're early-parallel work), WP-P1.11/13/14/15/16/17/18 (the other sub-commands, all parallel after their deps land).

12.2 Phase 2 critical path (Billet)

```
Ingot tagged
    → WP-P2.01 (postcard cache)
    → WP-P2.07 (HTTP client) → WP-P2.08 (extraction) → WP-P2.09 (minisign) → WP-
P2.10 (upstream flow) → WP-P2.12 (loran update)
    → WP-P2.13 (overlay merge)
    → WP-P2.02 (TUI shell) → WP-P2.03 (browse) → WP-P2.04 (detail) → WP-P2.05
(search)
    → WP-P2.18 (integration test suite)
        = Billet ready to tag
```

The TUI track and the update track are independent and can be worked in parallel after Ingot. The overlay-merge work is a third independent track.

12.3 Phase 3 critical path (Bloom)

```
Billet tagged
    → WP-P3.01 (full schema)
        → WP-P3.02 (MCP server) → WP-P3.03 (mcp sub-command wiring)
    → WP-P3.04 (DescribeIngestor, parallel with MCP)
    → WP-P3.07 (integration test suite)
        = Bloom ready to tag
```

WP-P3.05 (key rotation docs) and WP-P3.06 (cross-distro overlay) are off the critical path and can run in parallel.

---

13. Definition of Done (Per Phase)

13.1 Ingot — Definition of Done

Tag `v0.x` (pre-1.0 release of Phase 1) requires:

- [ ] All Phase 1 WPs (P1.01 through P1.19) acceptance criteria checked off.
- [ ] All Ingot-tagged FRs from PRD §8 pass integration tests.
- [ ] NFRs in PRD §9.1 (performance), §9.2 (memory safety), §9.4 (security — minus signing), §9.5 (privacy), §9.6 (POSIX), §9.8 (licensing) met.
- [ ] Standard §14 compliance checklist run; result in `compliance-log.md`.
- [ ] CHANGELOG entry written.
- [ ] All Tier 1 platforms build clean in CI.
- [ ] Bench suite passes on the release commit.
- [ ] At least 25 curated pages in the bundled tree.

13.2 Billet — Definition of Done

Tag `v1.0` requires:

- [ ] All Phase 1 + Phase 2 WPs acceptance criteria checked off.
- [ ] All FRs tagged Ingot or Billet pass integration tests.
- [ ] NFRs in PRD §9.3 (reliability) and §9.7 (accessibility) also met.
- [ ] First signed upstream tarball published.
- [ ] Bravais default install includes a Loran catalog with ≥80% coverage (M-01).
- [ ] All 20 legacy tools listed in M-02 are reverse-lookupable.
- [ ] Standard §14 compliance checklist run.
- [ ] CHANGELOG entry written.
- [ ] All Tier 1 platforms build clean in CI.
- [ ] Bench suite passes on the release commit, confirming all NFR-001 to NFR-005.

13.3 Bloom — Definition of Done

Tag `v1.x` (where x ⩾ 1) for Bloom:

- [ ] All Phase 1 + Phase 2 + Phase 3 WPs acceptance criteria checked off.
- [ ] All FRs from PRD §8 pass integration tests.
- [ ] All NFRs from PRD §9 met.
- [ ] MCP server invokable by Claude Code, Codex CLI, and Cursor with no special configuration.
- [ ] `DescribeIngestor` ingests at least 3 other Steelbore CLIs in test (M-07).
- [ ] At least 3 Steelbore CLIs reference `loran show <self>` in their `--help` (M-08).
- [ ] Open Question 1 (minisign key rotation) resolved and documented.
- [ ] Standard §14 compliance checklist run.
- [ ] CHANGELOG entry written.

---

14. References

14.1 Loran-internal

- `loran-spec-v0_2.md` — Canonical technical specification.
- `loran-prd-v0_1.md` — Product Requirements Document.
- `README.md`, `NOTICE.md`, `CONTRIBUTING.md` — Posture files (Standard v1.1 §5.2).
- `AGENTS.md`, `CLAUDE.md`, `SKILL.md` — Agent context files.
- `OPERATIONS.md` — Runbook (created in WP-P3.05).
- `BENCHMARKS.md` — Benchmark-suite documentation.
- `PUBLISHING.md` — Publisher pipeline contract (created in WP-CC.07).
- `compliance-log.md` — Standard §14 audit log.
- `CHANGELOG.md` — Release history.

14.2 Steelbore standards & skills

- **The Steelbore Standard v1.1** — Naming, priorities, license, posture, platform, PFA, key bindings, palette, fonts, UI/UX, time, attribution.
- **Steelbore SFRS v1.0.0** — Dual-Mode Self-Documenting CLI Framework.
- **Steelbore Agentic-CLI Standard** — Agentic surface conventions.
- **Steelbore Rust Guidelines** — Crate choices, `unsafe` policy, error handling.

14.3 External

- tldr-pages: `https://tldr.sh/`
- minisign: `https://jedisct1.github.io/minisign/`
- ratatui: `https://ratatui.rs/`
- XDG Base Directory Specification.
- JSON Schema Draft 2020-12.

- Model Context Protocol: `https://modelcontextprotocol.io/`.

Forged in Steelbore.